Resource-Constrained Odometry, Localization, and Mapping on a Human-Carrying Hexapod

Daniel González Lázaro

***Abstract*— This report summarizes the final implemented odometry, localization, and mapping stack for a human-carrying Klann-linkage hexapod built around a Jetson Nano, a RealSense D415, six RS485 servo motor drivers with 17-bit motor encoders, 50:1 planetary gearboxes, and an ESP32 micro-ROS bridge. telemetry, a ROS 2 C++ motor-state observer that unwraps single-turn 17-bit motor encoders through the 50:1 gearbox, a Klann-body kinematic odometry estimator, a planar weighted odometry fusion node, and an ORB-SLAM2 CUDA RealSense UDP bridge that exports pose, map points, and a lightweight virtual scan for `slam_toolbox` and Nav2 occupancy-grid navigation. The biggest embedded issues were not the high-level ROS interfaces, but the RS485 transaction schedule, the semantics of the servomotor encoder, and the limited reliability of the ESP32 ADC channels for foot-contact sensing. The final system produced teleoperated straight-line odometry results within the 6 tolerance required by the AXIS Dance Company collaboration, although the kinematic scale remains a tunable calibration parameter. The resulting architecture treats RGB-D SLAM as the primary mapping/localization source and treats body-kinematic odometry as a fallback and fusion component when visual tracking is degraded or the camera is partially occluded by riders or nearby people.**

I. INTRODUCTION AND FINAL SYSTEM CONTEXT

The target platform is an assistive mobility device for AXIS Dance Company that must transport a rider without wheels as the primary locomotor, fit within a 5 ft by 5 ft footprint, carry up to a 170 lb rider while keeping the total mass of the system below 150 lb, and navigate to a designated point in a performance stage within 6 in in 1 min [1]. The final demonstrations also require straight and curvilinear forward and backward traversals over 10 feet in a test area of 10 feet by 10 feet, with penalties for path deviation and manual intervention [1]. These requirements make odometry and mapping central to the project rather than optional instrumentation.

The final implementation uses a staged estimator. The RealSense D415 and ORB-SLAM2 CUDA process provide exteroceptive motion and sparse mapping; the ROS 2 UDP bridge converts this output into ROS odometry, point-cloud visualization, and a virtual planar scan for `slam_toolbox` occupancy mapping. That occupancy grid is included for the Nav2 navigation stack, not merely for visualization: it is the grid map on which future autonomous goal selection, costmaps, and local planning can operate. In parallel, ESP32 reads motor-driver encoder data and publishes compact six-motor telemetry. An ROS 2 C++ observer converts these raw encoder counts into a continuous crank phase, and the Klann linkage model estimates body twist from stance-foot kinematics. A weighted fusion node then provides the planar `odom -> base_link` transform used by mapping and visualization.

The system is intentionally conservative. Roll and pitch are kept out of the 2D mapping transform so that `slam_toolbox` receives a planar `map -> odom -> base_link` tree. IMU data remain available, but the default mapping path uses only planar yaw-rate feedback and avoids injecting absolute IMU roll, pitch, or unverified yaw into the map frame. This was necessary because early versions produced visually plausible local motion but nonsensical tilted frames in RViz, which made the sparse ORB map and occupancy map appear rotated relative to the robot.

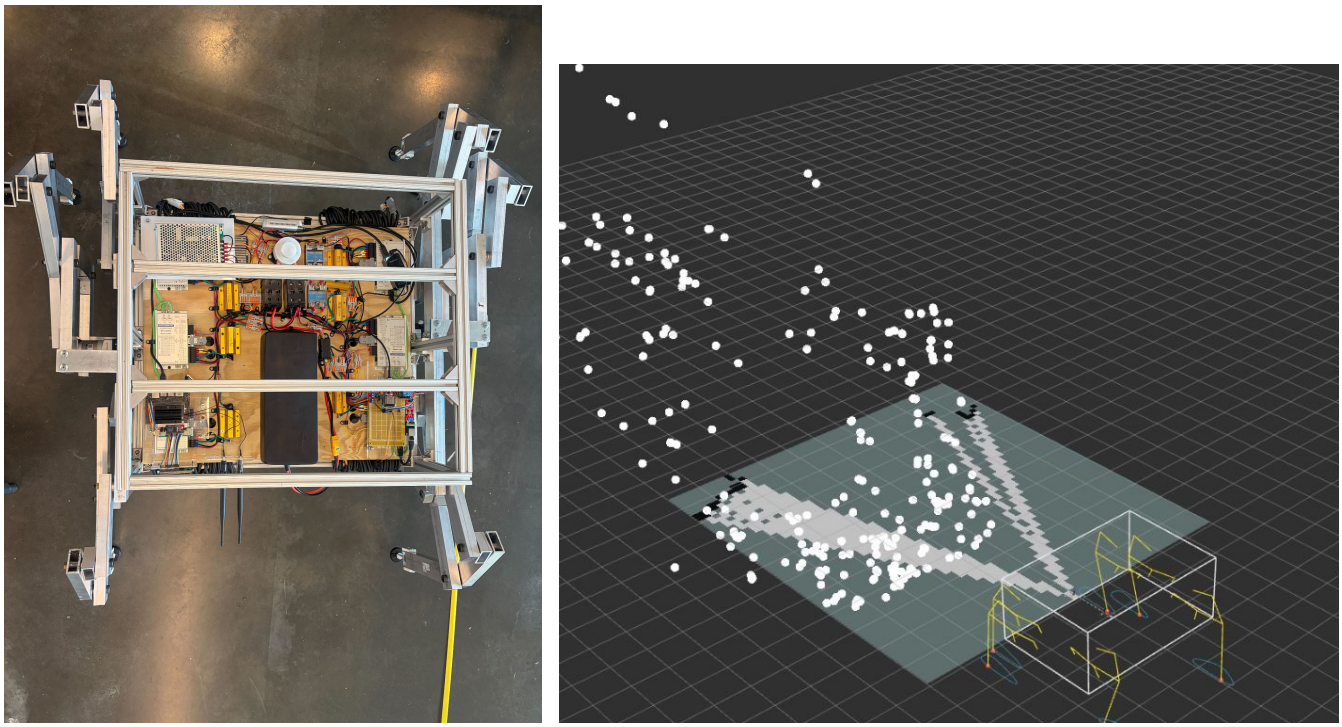
A practical reason for keeping the leg-based odometry path strong is that it cannot be assumed that the camera has a clean forward view during public interaction. Riders may naturally mount the robot with their legs hangingcolcon build –symlink-install –packages-select hexapod$_{o}perator_{i}nterface\ hexapod_{n}av2_{b}ringupcolconbuild-$
$-symlink - install - -packages -$
$select\ hexapod_{o}perator_{i}nterface\ hexapod_{n}av2_{b}ringupcolconbuild-$
$-symlink - install - -packages -$
$select\ hexapod_{o}perator_{i}nterface\ hexapod_{n}av2_{b}ringupcolconbuild-$
$-symlink - install - -packages -$
$select\ hexapod_{o}perator_{i}nterface\ hexapod_{n}av2_{b}ringupalongthesides,$
$onlyodometryalreadysatisfiesthecompanynavigationtoleranceintel$
$linetests, laterautonomousiterationscanreprioritizetheD415stream$

II. FINAL EMBEDDED MOTOR AND ENCODER IMPLEMENTATION

The final ESP32 firmware accepts `dsy_motor_msgs/MotorRpmArray` on `/motor_rpm_cmd` and publishes `dsy_motor_msgs/MotorOutputOdometryArray` on `/motor_output_odom`. The message package name is historical; the relevant hardware behavior is that one ESP32 controls six RS485 servo drives. The six-element command and odometry vectors remain in physical ESP bus order `[rb, rm, rf, lf, lm, lb]`, which follows the physical wire distance from the ESP32 to the motor drivers on the RS485 daisy chain, from closest to farthest. On the host, this is remapped to the canonical kinematic model order `[rf, rm, rb, lf, lm, lb]` through `[2, 1,`

This implementation was a component of Spring 2026 mechatronic design's team 2, and our human-carrying hexapod project. Other members of team 2 are: Yechen Zheng, Hongfei Liu, and Hongru Liu

Project website: `https://mechatronic-design-s2026-team-2.github.io/Website/`. Code repository: `https://github.com/Mechatronic-Design-S2026-Team-2/Design`.



(a) final top-view hardware layout

(b) merged RViz view with skeleton, ORB points, and occupancy/grid products

Fig. 1. Final physical and mapping configuration. The top-view chassis photograph shows the distributed electronics, motor-driver layout, RS485 wiring context, and six linkage modules. The merged visualization shows the robot skeleton, sparse visual map products, and the grid/scan products used to validate scale and transform consistency before using the map for Nav2 navigation.

`0, 3, 4, 5]`. This separation was important because bus-debugging tools, direct teleoperation tests, and firmware logs are easiest to interpret in cable order, while the Klann model and controller need a consistent front/middle/back order for each side.

The final embedded code deliberately separates startup configuration from runtime communication. Startup retains the slower verified path for motor-driver configuration, including control mode, torque/current limiting, speed command source, speed limits, acceleration/deceleration parameters, and enable sequencing. Runtime uses short RS485 helpers for changed RPM writes and encoder reads. This reduced risk because earlier event-driven versions changed too much of the bringup code at the same time as the runtime path. Reusing the proven power/enable sequence also avoided failures where the active-low servo-enable relay briefly left the drives disabled or caused inconsistent drive readiness.

The runtime message includes timing fields added specifically for delay-aware host estimation: `command_seq`, `last_cmd_seq`, `esp_cmd_rx_time_us`, `esp_apply_complete_time_us`, `esp_poll_complete_time_us`, and `esp_publish_time_us`. These fields allow the ROS 2 observer to estimate command receive-to-apply time, apply-to-poll time, poll-to-publish time, and the external transport delay between the host and ESP32. The estimator uses these values to compensate phase for measurement age and future command-application delay rather than treating every motor sample as instantaneous.

TABLE I

FINAL DEPLOYED RUNTIME PARAMETERS USED BY THE SOFTWARE STACK.

Parameter	Final value
RS485 motor bus baud rate	115200
micro-ROS UART baud rate	921600
RS485 runtime response timeout	50 ms
RS485 runtime turnaround delay	2 ms
Runtime transaction attempts	1
Six-drive encoder sweep period	25 ms
Odometry publish period	50 ms
Encoder counts per motor rev	131072
Gearbox ratio	50:1
Driver accel/decel time	250 ms / 250 ms
Kinematic XY scale	2.1786
ORB/scan scale	1.382
Virtual scan range	0.45–3.0 m
Camera X offset from `base_link`	0.2794 m

The motor command path uses changed-only writes by default. Once a drive has accepted a commanded RPM, that RPM is not resent unless the value changes or a periodic resend mode is explicitly enabled. This behavior matters on a six-drive 115200 baud bus. Continuously rewriting unchanged commands consumes the same bus capacity needed to poll encoders, and it can make the odometry stream appear frozen even though the micro-ROS topic itself is still publishing.

The key firmware change is that the ESP32 no longer

attempts to compute output-shaft phase in the embedded runtime path. Instead, `output_phase_rad[]` is reused, for compatibility with the existing message definition, to carry raw P18.32/P18.33 single-turn encoder counts. For the installed M17S 17-bit encoders, the raw range is 0 to 131071 counts per motor revolution, which fits exactly in the float field used by the current message. The host computes continuous phase by unwrapping the modulo count, applying the 50:1 gearbox ratio, applying the physical-to-model sign convention, and subtracting the startup reference. This moved the ambiguous part of the estimate from the memory- and timing-constrained ESP32 into the ROS 2 observer, where command history and model constraints are available.

The code finally treats P18.32/P18.33 as a single-round absolute encoder value and leaves P18.34 out of the odometry path. Earlier debugging considered feedback pulse counters and multi-round monitor registers, but the reliable source for the installed M17S encoder was the modulo single-turn value. The RS485 driver also includes a word-order guard for this 32-bit monitor value: if the high-word-first interpretation is outside the expected 17-bit range but the low-word-first interpretation is valid, the valid ordering is selected. This avoids silently accepting a nonsensical encoder count caused by Modbus word-order differences.

For a raw count c_k, counts per motor revolution $N = 131072$, and gearbox ratio $G = 50$, the host observer estimates output phase as

$$\phi_k = s_i \frac{2\pi}{NG} \hat{C}_k + \phi_{0i}, \tag{1}$$

where s_i is the physical-to-model sign for leg i, ϕ_{0i} is the configured phase offset, and $\hat{C}_k$ is the continuous unwrapped motor-count estimate. The observer predicts $\hat{C}_k$ from the latest accepted command and the configured acceleration/deceleration time, then corrects it toward the raw encoder measurement by choosing the nearest plausible wrap. A bounded innovation prevents a single stale or misread RS485 sample from jumping the phase by multiple motor turns.

This design also makes explicit that the encoder is a single-turn motor encoder, not a multi-turn output encoder. A full output-shaft revolution corresponds to 50 motor revolutions, so the raw motor encoder wraps many times during one output cycle. The host can infer the correct wrap count only if the raw count is polled often enough that the count change between accepted samples remains below the half-revolution ambiguity limit, or if commanded RPM and acceleration constraints are used to choose the nearest plausible wrap. The final observer therefore combines raw-count measurements with a ramp-limited command prediction. It ignores unchanged raw samples during commanded motion when the ESP has merely republished a cached six-element vector while the RS485 worker has not yet refreshed that drive.

The physical phase zero was set manually. Before tests, each linkage output crank was positioned to point straight down from the motor output shaft so that all six legs began from the same mechanical phase. In software, this corresponds to zero phase offsets in the deployed configuration. This manual setup made phase visualization, tripod timing, and straight-line odometry calibration much easier because startup phase differences were dominated by actual encoder readings rather than arbitrary assembly offsets.

III. RS485 TIMING ISSUES AND FINAL SCHEDULER

The most important embedded problem was RS485 bus scheduling. The system has one ESP32 talking to six RS485 servo motor drivers over a 115200 baud bus. Every transaction requires bus turnaround time, response timeout handling, and drive-specific acknowledgement. Early runtime designs either wrote repeated unchanged RPM commands, interleaved reads and writes too aggressively, or polled only one drive per publish cycle. Those versions could appear responsive for one tripod group while starving or freezing the other group, especially when commands were updated faster than the bus could complete the read/write sequence.

The observed failure modes were consistent with scheduler starvation rather than ROS topic failure. Some versions produced a latching tripod update pattern: one set of legs would update after one unique command, then the opposite set would update only after another unique command. Other versions updated only one phase per published message, which made the ROS topic appear alive but too sparse to support continuous crank unwrapping. Increasing transaction retries could make more values update, but it also increased bus occupancy and stop latency. The final design therefore focused on deterministic coverage rather than brute-force retrying.

The final firmware separates startup configuration from runtime communication. Startup still uses the slower, verified drive configuration path for mode setup, torque limits, enable behavior, speed command source, speed limits, and acceleration/deceleration parameters. Runtime uses shorter RS485 read/write helpers with menuconfig-exposed timing values. The final defaults include a 115200 baud RS485 bus, 921600 baud micro-ROS UART, a 50 ms runtime response timeout, a 2 ms runtime turnaround delay, and one runtime transaction attempt. These values were chosen to avoid indefinite pending-write behavior while keeping operator stop latency acceptable.

The runtime worker is a strict six-drive encoder sweep state machine. A new sweep starts at the configured sweep period, reads drive 1 through drive 6 without artificially spacing the reads by the main worker loop delay, and then waits until the next sweep period. A pending all-zero RPM command preempts the sweep between RS485 transactions, so a stop command is applied before additional noncritical encoder reads. A pending nonzero command is applied when no sweep is active. Unchanged RPM values are not rewritten unless an explicit resend mode is enabled, reducing bus pressure. This scheduler was the major change that made the firmware practical: it preserves full six-drive odometry coverage while preventing the bus from being consumed by continuous single-drive polling.

The remaining timing caveat is inherent to single-turn encoders. Higher motor RPM reduces the time available before one motor-encoder wrap becomes ambiguous. At 1000 motor rpm, a motor revolution lasts about 60 ms; with a 50:1 gearbox, the output is only 20 rpm, but the encoder still wraps every motor revolution. Therefore the useful odometry limit is not only the mechanical output speed; it is the RS485 polling rate relative to motor-encoder wrap rate. The final host estimator mitigates this using command prediction, acceleration/deceleration timing, and an innovation limit, but no software observer can fully recover a wrap count if the bus misses too many consecutive raw samples during high-speed motion.

IV. HOST-SIDE PHASE, BODY-KINEMATIC ODOMETRY, AND FUSION

The ROS 2 C++ workspace implements the host-side estimator in `hexapod_hardware_cpp`. The `motor_state_aggregator_node` subscribes to `/motor_output_odom` and to direct `/motor_rpm_cmd` commands. It keeps command sequence history, ESP-domain timestamps, and host receive times to estimate command propagation delay. It then unwraps the raw single-turn encoder count into continuous motor counts, converts counts to output crank phase using 131072 counts per motor revolution and the 50:1 gearbox, and publishes `/hexapod/motor_state`. Phase velocity is computed as a weighted blend of measured phase difference and command-predicted phase rate.

The `body_state_estimator_node` evaluates the measured Klann linkage model for each leg. For each stance leg, the no-slip constraint is that body translational velocity plus yaw-induced foot velocity plus the foot velocity from the linkage should be approximately zero. The node solves a weighted least-squares problem for planar body twist:

$$\begin{bmatrix} 1 & 0 & -p_{iy} \\ 0 & 1 & p_{ix} \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} = - \begin{bmatrix} \dot{p}_{ix} \\ \dot{p}_{iy} \end{bmatrix}, \tag{2}$$

for all legs whose stance confidence exceeds the configured threshold. At least three stance legs are required before the estimate is trusted. This matches the physical intent of the alternating tripod gait: while three feet are supporting the robot, their foot velocities constrain body motion.

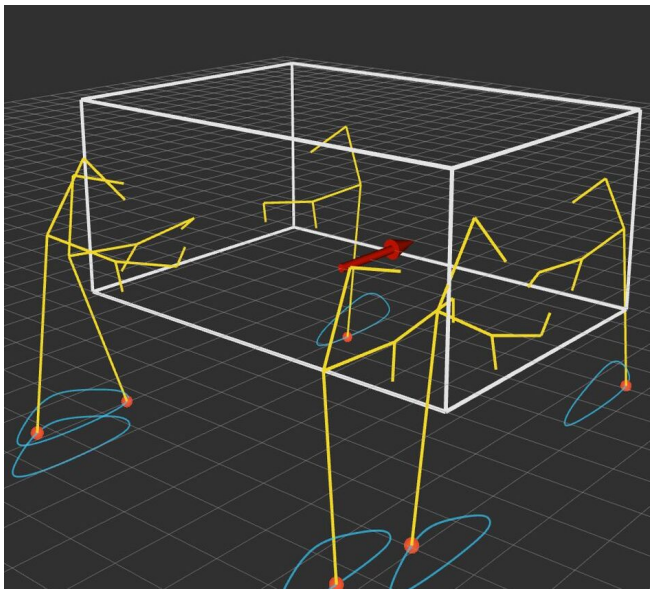
The kinematic model uses the measured linkage YAML rather than hard-coded foot trajectories. This is important because a Klann leg is not a simple wheel or crank-slider. One rotating output crank drives a set of bars that produce an approximate walking foot path: part of the cycle is a low, nearly horizontal stance stroke that can push against the ground, and the rest is a lifted return stroke. The odometry problem is therefore to infer body motion from six measured crank phases and the modeled velocity of each foot tip, not from wheel radius or direct linear encoders.

Each leg is solved in its local sagittal plane. The YAML describes the motor-shaft location, fixed top and bottom pivots, crank radius, link lengths, rigid angle constraints, unknown joint names, and foot point. For a given crank phase, the solver first places the crank tip and then solves the nonlinear linkage constraints for the remaining joints. The implementation uses a damped least-squares solve over distance residuals and angle residuals, with the previous phase's solution reused as the next initial guess. The local foot coordinate is then mounted into the robot body frame with the measured shaft position and lateral offset, giving the stance-foot position used by the body-twist solve.

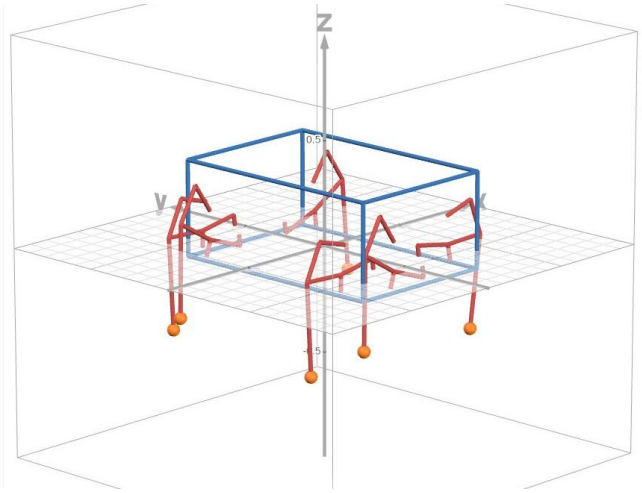
The same geometry is shared by the visualizer, the body-state estimator, and the phase controller. For deployment, however, repeatedly solving the nonlinear linkage at runtime is undesirable on the Jetson Nano because SLAM, virtual-scan generation, micro-ROS, TF publication, and visualization are already competing for CPU. The workspace therefore includes a configurable lookup-table generator that precomputes each leg's phase-to-foot trajectory on a faster computer. The table is indexed by output-crank phase, not by the raw motor encoder count. This distinction is important: the encoder has 131072 counts for one motor-input revolution, while the 50:1 gearbox means one output-crank revolution spans 6553600 possible motor-count states. A lookup table that exactly matched the encoder-derived output-phase lattice would therefore be unnecessarily large and would not improve accuracy beyond the mechanical model, backlash, foot compliance, and slip errors. The practical table resolution should be selected from interpolation error in the smooth Klann trajectory rather than from the encoder's single-turn count. For this project, 8192 to 16384 samples per output-crank revolution is the appropriate production range, with larger tables such as 32768 or 131072 samples useful only for offline validation of interpolation error and stance-window boundaries. The generator writes one compact binary `.klut` file per leg plus a patched YAML file that points the runtime system at those tables.

Each lookup sample stores local foot position, numerical derivatives with respect to output-crank phase, and stance confidence. The derivative is used to convert measured phase velocity into foot-tip velocity without running a second linkage solve. Stance confidence combines the configured stance phase window with geometric information from the trajectory: low foot height and small vertical motion are weighted more heavily than lifted return motion. Runtime evaluation then becomes interpolation between adjacent table entries. This converts the body odometry path from an iterative geometry solve into a deterministic table lookup, which is better suited to Nano-class compute and makes the estimator's timing more predictable. The interpolation error can be bounded by regenerating the table at multiple resolutions and comparing the interpolated foot position and derivative against the direct nonlinear solve; that check is a better criterion than matching the input encoder resolution.

The host model includes delay compensation but is not a full Kalman filter. The motor-state message carries estimated measurement age and future command-application delay. The body estimator advances each measured phase by the estimated measurement age times the latest phase velocity



(a) ROS/RViz skeleton and modeled foot paths



(b) Desmos 3D linkage model used for geometry validation

Fig. 2. Klann-linkage modeling views. The runtime estimator uses the same linkage geometry represented in the ROS visualizer and the public Desmos model. The Desmos model is available at `https://www.desmos.com/3d/18nie5ecgp`.

before evaluating the linkage model. This command-preview approach was simpler to debug than a per-leg EKF while the RS485 timing and encoder semantics were still being stabilized. A per-leg EKF remains a reasonable follow-up. Such a filter would estimate each leg's continuous motor count, velocity, possibly acceleration or driver-latency bias, and wrap index using commanded RPM as the process input and the modulo 17-bit encoder count as the measurement. Its benefits would be explicit uncertainty per leg, smoother behavior through missed RS485 samples, principled rejection of stale or misordered reads, and a cleaner way to fuse FSR contact probability when the analog hardware is stabilized. The tradeoff is tuning and debugging cost: process noise must represent driver acceleration limits, RS485 latency, and contact/slip uncertainty; the discrete wrap ambiguity still cannot be recovered if polling violates the half-motor-revolution limit for too long; and an EKF can hide timing bugs that are easier to diagnose with the deterministic unwrap currently used. Since the deterministic observer already met the straight-line tolerance, the EKF should be treated as a reliability and uncertainty-improvement path rather than a required class-final component.

After straight-line testing, the final code includes a kinematic mirror and axis-scale calibration. In the 73 in test, the raw kinematic estimate before the final scale adjustment reported approximately 0.851 m of forward motion for a physical 1.854 m displacement. The final configuration therefore enables `mirror_kinematics_across_y_axis` and applies `body_twist_axis_scale: [2.1786, 2.1786, 1.0]`. This produced teleoperated straight-line odometry within the 6 in task tolerance, but the scale should still be treated as an empirical calibration parameter. A larger data set with forward, backward, and turning trials should be used before treating the coefficient as final.

The `weighted_odom_fusion_node` publishes the generic `/odom` and the `odom -> base_link` transform. It can fuse kinematic odometry, IMU data, and ORB-SLAM odometry. In the mapping configuration used here, the node forces the pose planar, keeps roll and pitch weights at zero, and fuses ORB-SLAM as a relative planar odometry source. The first ORB pose becomes the local origin so that a large absolute ORB yaw at startup does not rotate the robot skeleton or the occupancy grid. This makes the leg-kinematic odometry useful both as a fallback when ORB tracking is lost and as a fusion component when ORB-SLAM is available but intermittently noisy.

The final TF ownership is explicit. `slam_toolbox` owns `map -> odom`. The weighted fusion node owns `odom -> base_link`. The ORB-SLAM UDP bridge publishes `/hexapod/orbslam_odom`, `/orbslam/map_points`, and `/scan`, but it does not publish a competing `odom -> base_link` transform in the fused launch configuration. The bridge also publishes or launches the static transform from `base_link` to `orbslam_scan_frame`, using the measured 11 in, or 0.2794 m, forward camera offset.

This ownership split fixed several practical visualization problems. When both ORB-SLAM and the kinematic stack attempted to own the same odometry transform, the robot skeleton could move in a different frame than the occupancy grid. When ORB-SLAM packets were converted from camera optical axes twice, the camera-forward direction appeared as the wrong robot axis and points that should have appeared in front of the robot appeared above it. When packet timestamps from the bare-metal ORB-SLAM process were used inside the container without synchronized

clocks, slam_toolbox and RViz could drop scans due to TF extrapolation. The final bridge disables the second axis conversion, uses local receive stamps by default, and can stamp scans with the latest pose stamp.

V. ORB-SLAM2 CUDA, UDP BRIDGE, AND OCCUPANCY MAPPING

The ORB-SLAM2 CUDA RealSense package was modified so that it is more robust in the deployed Jetson workflow. The RealSense pipeline now retries startup, restarts after repeated frame timeouts, validates aligned color and depth frames before tracking, clones frame memory before handing images to ORB-SLAM2, and skips malformed frames instead of passing empty or mismatched matrices into the tracker. These changes were made to reduce runtime crashes and segfault-like failures caused by transient camera, alignment, or frame-buffer issues.

The deployed ORB-SLAM2 process is run by a systemd service on the Jetson outside the ROS 2 container so that the CUDA-enabled ORB-SLAM2 build and CUDA/OpenCV library paths remain under direct system control. The service runs the RealSense RGB-D executable at 640 by 480 and 30 Hz, sends UDP packets to 127.0.0.1:5005 when the ROS 2 container uses host networking, restarts on failure, and uses SIGINT for clean shutdown. Environment variables enable UDP pose, sparse map chunks, ROS-axis output, and the depth-derived scan stream.

The RGB-D input path was also corrected for metric consistency. Depth is aligned to the color stream every frame, converted from RealSense Z16 units to CV_32FC1 meters, and paired with YAML files using DepthMapFactor: 1.0. This avoids mixing unscaled integer depth units with meter-scale assumptions. The UDP output path can publish pose and map points in ROS axes by default, using +x forward, +y left, and +z up. On the ROS 2 side, the bridge default disables a second axis conversion to avoid double-transforming the ORB output.

The ROS 2 UDP bridge publishes /hexapod/orbslam_odom, /orbslam/map_points, and a lightweight virtual /scan. The virtual scan is extracted from a horizontal depth band, decimated to reduce CPU load, and then consumed by slam_toolbox to produce the 2D occupancy grid intended for Nav2 navigation. The bridge also applies the measured 11 in forward camera offset from base_link. The final launch defaults use a position and scan-range scale of 1.382, scan publish decimation of 3, packet stamps disabled to avoid host/container clock skew, and scan stamps tied to the latest pose stamp to avoid TF extrapolation drops. On the ORB-SLAM2 sender, the service uses 240 scan rays, a 24-pixel vertical band, and a 0.45–3.0 m range window. Sparse map chunks are decimated before transmission and limited to small point packets so that map visualization does not starve pose and scan traffic.

A key lesson is that the sparse ORB map point cloud is not itself the occupancy map. ORB-SLAM2 provides sparse landmarks and camera motion, while slam_toolbox constructs the occupancy grid from the virtual scan. Keeping those products separate made debugging much easier: ORB map points were used to verify axes and visual scale, while the occupancy grid was used to evaluate whether the robot could support Nav2 navigation goals and later costmap-based obstacle avoidance.

VI. FOOT CONTACT SENSING AND FSR LIMITATIONS

The intended leg odometry model included foot-ground contact from FSRs so that stance legs could be selected from hardware contact measurements rather than inferred entirely from linkage phase. In practice, assessing FSR effectiveness was difficult with the available ESP32 ADC inputs. The ADC readings swung widely enough that repeatable, verifiable contact state was not available during the final software integration period. Because a false contact can corrupt the no-slip body-twist solve as badly as a missed contact, the final deployed body-kinematic estimator relies primarily on phase-derived stance confidence from the Klann model.

This does not mean FSRs are ineffective for the robot; it means this particular hardware and software path was not yet calibrated enough to trust them as odometry gates. A useful follow-up would add better analog conditioning, stable voltage references, shielded routing away from motor noise, per-sensor calibration under known loads, and software hysteresis with debounced contact state. Once the ADC path is stabilized, FSRs should improve the estimator because they can distinguish true stance from unloaded swing or slip conditions. Until then, the encoder phase is already accurate enough to track tripod gait phase, provided the RS485 polling rate stays fast enough to avoid single-turn wrap ambiguity.

The final software therefore treats FSRs as a future contact-quality improvement, not as a required source for odometry. The kinematic estimator already has a phase-derived stance confidence from the Klann trajectory. Hardware contact would improve robustness during slip, uneven loading, or gait transitions, but only after the contact state is repeatable enough that it does not introduce false stance constraints. This was a deliberate safety decision: an untrusted contact signal can make dead-reckoned odometry worse than a purely model-based stance estimate.

VII. EVALUATION RESULTS AND REMAINING LIMITATIONS

The final teleoperation tests showed that the body-kinematic odometry can satisfy the project tolerance for straight-line motion after calibration. The main quantitative calibration trial moved the robot 73 in, or 1.854 m. The uncalibrated kinematic odometry reported approximately 0.851 m in the forward axis with very small lateral drift, which led to the final body_twist_axis_scale value of 2.1786. After applying the final sign, mirror, and scale corrections, the teleoperated straight-line odometry error was within the 6 in tolerance established for the dance-company

collaboration. The reported orientation for that run corresponded to only a small yaw error, approximately 0.033 rad, or about 1.9 deg, during the straight traverse.

This result changes the role of the camera in the system architecture. Since the encoder-plus-linkage estimate is sufficient for the demonstrated navigation tolerance, the camera does not have to be the only source of motion continuity. In autonomous forward iterations, the RGB-D stream can be weighted more heavily toward detecting obstacles, people, and free space for Nav2 costmaps, while kinematic odometry maintains short-term body motion. This is especially relevant for the intended public-use context: a rider's natural posture can leave legs hanging beside the robot, and those legs can partially block the forward D415 view even though the drivetrain and body controller still need a stable odometry source.

ORB-SLAM scale was also tuned empirically. An earlier 52 in test led to an ORB/scan scale of approximately 1.214. The later 73 in test, in the final launch context, implied a total ORB/scan scale of 1.382. The bridge applies this same scale to pose, sparse map points, and laser scan ranges so that visual odometry and occupancy geometry remain mutually consistent. This is a practical calibration, not proof that all depth and SLAM scale errors are resolved. Additional tape-measured tests should be performed with the final camera exposure, resolution, and depth-filter settings.

This result should be interpreted with care. It demonstrates that the encoder-plus-linkage model is useful and that the final software architecture is directionally correct. It does not prove that the model is globally calibrated for all motions. Curvilinear motion, backward motion, payload-dependent body oscillation, foot slip, and nonideal stance timing can all change the effective body displacement per crank phase. Therefore, the final system should retain both SLAM and kinematic odometry rather than replacing one with the other. The most practical long-term estimator is a fused system in which ORB-SLAM constrains global drift when visual tracking is good, while kinematic odometry supplies continuity through short visual failures, dynamic-scene corruption, or temporary loss of mapped features.

The remaining limitations are primarily calibration and robustness issues. The kinematic model still needs more tape-measured trials to separately tune the scale for forward, reverse, and yaw motion. The RS485 polling period must remain related to the commanded RPM limit because higher motor speed increases wrapping ambiguity. The FSR hardware path requires analog and software filtering before it can be used as a reliable contact source. Finally, ORB-SLAM2 CUDA is more robust after RealSense and UDP hardening changes, but remains a backported package on constrained hardware; production use should still include process supervision, logging, and a graceful fallback to kinematic odometry.

It is possible that AXIS Dance Company selects this hexapod design for future performances because it was the only S26 hexapod design that succeeded in either navigation or locomotion. That prospective use leaves substantial engineering work: verifying the system in real stages, measuring reliability over longer demonstrations, validating performer-interaction behavior, and tuning locomotion for speed, smoothness, and rider comfort. Mechanical follow-up work could include changing leg orientations, revisiting gait phase offsets, and replacing the six-bar Klann linkage with eight-bar or higher-order walking links whose foot paths can provide flatter stance trajectories and lower vertical body excitation. In terms of hardware selection, moving to reliable higher baud communication like CANbus or multi-turn encoder servos in such circumstances where lead times and instant part availability is less of a concern would also add substantial reliability margins for leg estimation and control, freeing up computational resources on the onboard computer for body optimization and camera-based performer avoidance.

VIII. CONCLUSION

The final code achieved a central project objective: deployable embedded odometry and mapping path for the human-carrying hexapod, enabling it to have the capacity of accurately meeting distance and velocity targets as specified by the project design requirements, either by velocity teleoperation or waypoint navigation by off-stage operators. The ESP32 firmware now minimizes RS485 bus pressure, prioritizes all-zero stop commands, and exports raw single-turn encoder counts rather than fragile embedded phase estimates. The ROS 2 host stack unwraps these encoder counts into continuous output crank phase, computes Klann body-kinematic odometry from stance-foot constraints, and fuse this estimate with ORB-SLAM2-derived odometry in a planar TF tree suitable for Nav2 occupancy mapping. The ORB-SLAM2 CUDA package and ROS 2 UDP bridge were hardened for RealSense frame reliability, metric depth handling, ROS-axis consistency, scan generation and Jetson-class CPU limits. The resulting system met the 6 in straight-line teleoperation tolerance after calibration, which allows future camera processing to focus more on dynamic obstacle and people avoidance while still leaving clear follow-up work in kinematic scale tuning, FSR contact reliability, broader fused-odometry validation, and stage-readiness testing. Project resources are available through the team website [12], code repository [13], and Desmos linkage model [14].

REFERENCES

[1] Mechatronic Design Spring 2026, "Kinematic/Kinesthetic Dance Robot Specifications," Version 4.0, course handout.

[2] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, Present, and Future of Simultaneous Localization and Mapping: Towards the Robust-Perception Age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.

[3] K. Ebadi *et al.*, "Present and Future of SLAM in Extreme Underground Environments," arXiv:2208.01787, 2022.

[4] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[5] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[6] M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

[7] B. Bescos, J. M. Facíl, J. Civera, and J. Neira, "DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.

[8] C. Yu, Z. Liu, X. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, "DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1168–1174.

[9] W. Dai, Y. Zhang, P. Li, Z. Fang, and S. Scherer, "RGB-D SLAM in Dynamic Environments Using Point Correlations," arXiv:1811.03217, 2018.

[10] UZ-SLAMLab, "ORB-SLAM3," GitHub repository, accessed Apr. 7, 2026.

[11] B. Bescos, "DynaSLAM," GitHub repository, accessed Apr. 7, 2026.

[12] Mechatronic Design Spring 2026 Team 2, "Project Website," `https://mechatronic-design-s2026-team-2.github.io/Website/`, accessed Apr. 30, 2026.

[13] Mechatronic Design Spring 2026 Team 2, "Design Repository," `https://github.com/Mechatronic-Design-S2026-Team-2/Design`, accessed Apr. 30, 2026.

[14] Team 2, "Hexapod Klann-Linkage Desmos 3D Model," `https://www.desmos.com/3d/18nie5ecgp`, accessed Apr. 30, 2026.